JAVA 8 MASTER HANDBOOK

Lecture 1 – Java 8 Introduction

Part 2 – Functional Programming, Imperative vs Declarative Programming & Java 8 Philosophy

Goal of Part 2

Is part ke end tak tum ye confidently explain kar paoge:

- Functional Programming kya hoti hai?
- Java ne Functional Programming support kyun add ki?
- Imperative aur Declarative Programming me actual difference kya hai?
- Java 8 ki philosophy kya hai?
- Lambda Expressions ka foundation kya hai?

Table of Contents

Chapter 1 - What is a Programming Paradigm?

Sabse pehle ek basic question.

Programming Paradigm kya hota hai?

Definition

Programming Paradigm ka matlab hota hai:

> **Programming karne ka ek style ya approach.**

Jaise ek hi problem ko solve karne ke kai tareeke ho sakte hain.

Programming me bhi ek problem ko solve karne ke multiple styles hote hain.

Inhi styles ko Programming Paradigm kehte hain.

Real World Example

Suppose tum Delhi se Jaipur jana chahte ho.

Tumhare paas options hain.

```
Car

Bus

Train

Flight
```

Destination same.

Lekin journey ka style alag.

Programming me bhi exactly aisa hi hota hai.

Different Programming Paradigms

Sabse common paradigms.

```
Procedural Programming

↓

Object-Oriented Programming

↓
```

```
Functional Programming

↓

Event Driven Programming

↓

Logic Programming
```

Java primarily

```
Object-Oriented
```

language thi.

Java 8 ke baad

```
Object-Oriented

+

Functional Programming Features
```

Important Interview Question

Is Java a Functional Programming Language?

Answer

NO

Ye bahut important hai.

Interviews me candidates yahin galti karte hain.

Java

❌ Pure Functional Language nahi hai.

Java

✅ Functional Programming ko support karta hai.

Ye difference yaad rakho.

Why?

Pure Functional languages ke examples:

- Haskell
- Clojure
- Erlang

Ye languages functions ko true first-class citizens treat karti hain.

Java me aisa nahi hai.

Java me

- Lambda Expressions
- Functional Interfaces

ki help se function-like behavior represent kiya jata hai.

Isliye Java ko

> **Multi-Paradigm Language**

bolna zyada correct hai.

Interviewer's Note

Agar interviewer puche:

> **Is Java Functional Language?**

Kabhi mat bolna:

> "Yes"

Correct answer:

> Java is primarily an Object-Oriented Programming language. From Java 8 onwards it also supports Functional Programming features.

Ye answer senior interviewers ko impress karta hai.

Chapter 2 - What is Functional Programming?

Ab sabse important concept.

Definition

Functional Programming ek programming style hai jisme

> **Functions ko primary building block maana jata hai.**

Traditional programming me focus hota hai

```
Objects
```

Functional Programming me focus hota hai

```
Functions
```

Real Life Analogy

Suppose tum Swiggy use karte ho.

Tum order place karte ho.

Tum ye nahi batate:

```
Chef kaise banayega

Oil kitna dalega

Gas kitni jalayega
```

Tum sirf bolte ho:

```
Pizza chahiye.
```

System execution handle karta hai.

Exactly isi tarah Declarative Programming ka concept kaam karta hai.

Traditional Programming

Tum machine ko bolte ho

```
Step 1

↓

Step 2

↓

Step 3

↓

Step 4
```

Machine wahi karti hai.

Functional Programming

Tum sirf bolte ho

```
Mujhe Result chahiye.
```

Execution

System handle karta hai.

Why Functional Programming Was Introduced?

Ye sirf Java ka feature nahi tha.

Puri software industry change ho rahi thi.

1. Large Applications

Applications bahut badi hone lagi.

Millions of lines.

Maintenance difficult.

---

2. Duplicate Logic

Same loop

Same traversal

Same filtering

Again and again.

---

3. Better Readability

Old Code

```
Collections.sort(list,new Comparator<Employee>(){

    @Override

    public int compare(Employee a,Employee b){

        return a.getSalary()-b.getSalary();

    }

});
```

Java 8

```
list.sort((a,b)->a.getSalary()-b.getSalary());
```

Difference dekho.

---

4. Multi-Core Processors

Pehle computers

```
1 CPU
```

Aaj

```
4 Core
```

```
8 Core

16 Core

32 Core
```

Companies ko code parallel chalana tha.

Traditional loops parallel banana difficult tha.

Streams ne ye easy banaya.

Isliye Functional Programming parallel execution ke liye bhi important bani.

Characteristics of Functional Programming

Ek achha Functional Programming style generally:

- Chhote reusable functions
- Less mutable state
- Clear input → output
- Easy testing
- Better readability
- Easier parallel processing

> **Note:** Ye characteristics hain. Java inme se kai concepts support karta hai, lekin Java pure functional language nahi hai.

Chapter 3 - Java Before Functional Programming

Suppose tumhare paas Employee List hai.

Tumhe Salary > 50000 print karna hai.

Java 7

```
for(Employee e : employees){

    if(e.getSalary()>50000){

        System.out.println(e);

    }
```

```
}
```

Ab Department

```
for(Employee e : employees){

    if(e.getDepartment().equals("IT")){

        System.out.println(e);

    }

}
```

Again Age

```
for(Employee e : employees){

    if(e.getAge()>30){

        System.out.println(e);

    }

}
```

Again Designation

```
for(Employee e : employees){

    if(e.getDesignation().equals("Manager")){

        System.out.println(e);

    }

}
```

Observe.

Traversal

Same.

Loop

Same.

Sirf condition change.

Problem

Loop baar baar likhna pad raha hai.

Java developers ne bola

Why not

Loop ek baar likho.

Logic alag bhej do.

Ye hi Lambda ki foundation hai.

Chapter 4 - Imperative Programming

Definition

Imperative Programming me hum machine ko

HOW

batate hain.

Kaise karna hai.

Example

1 se 10 tak sum.

```
int sum = 0;

for(int i=1;i<=10;i++){

    sum += i;

}

System.out.println(sum);
```

Tumne machine ko bataya.

- Variable banao.
- Loop chalao.
- Increment karo.
- Add karo.
- Print karo.

Har step manually.

Ye hai Imperative Programming.

Another Example

List me even numbers print karo.

```
for(Integer num : numbers){

    if(num % 2 == 0){

        System.out.println(num);

    }

}
```

Again

Tum machine ko step by step instructions de rahe ho.

Characteristics of Imperative Programming

- Step-by-step instructions
- Explicit loops
- Mutable variables common
- Developer execution control karta hai
- HOW define hota hai

Chapter 5 - Declarative Programming

Declarative Programming me hum machine ko

WHAT

batate hain.

Kaise karna hai

Ye system decide karta hai.

Example

```
IntStream.rangeClosed(1,10)
         .sum();
```

Observe.

Loop kaha hai?

Nahi.

Variable?

Nahi.

Increment?

Nahi.

Tumne sirf bola

```
Sum chahiye.
```

Execution

Framework ne handle ki.

Another Example

Imperative

```
for(Integer num : numbers){

    if(num % 2 == 0){

        System.out.println(num);

    }
```

```
}
```

Declarative

```
numbers.stream()
       .filter(num -> num % 2 == 0)
       .forEach(System.out::println);
```

Code chhota bhi hua.

Readable bhi.

Maintainable bhi.

Imperative vs Declarative

Imperative	Declarative
HOW batata hai	WHAT batata hai
Manual loops	Stream operations
More boilerplate	Less boilerplate
Execution control developer ke paas	Execution framework handle karta hai
Usually longer code	Usually shorter code

Interview Trap

Interviewer:

> Streams me loop kaha hota hai?

❌ Galat answer:

> Stream me loop nahi hota.

✅ Correct answer:

> Stream internally iteration karta hai. Developer explicit loop nahi likhta, lekin iteration internally hoti hai.

Ye bahut important distinction hai.

Java 8 Philosophy

Java 8 ka core idea tha:

```
Collection

↓

Stream

↓

Operations

↓

Result
```

Aur behavior ke perspective se:

```
Behavior

↓

Lambda Expression

↓

Functional Interface

↓

Execution
```

Ye dono flow Java 8 ki backbone hain.

Real Industry Example

E-commerce website me requirement:

- Price > 1000
- Rating > 4
- Category = Electronics
- Sort by price

- Top 10 products

Imperative style me multiple loops aur sorting logic likhna pad sakta hai.

Declarative Java 8 style:

```
products.stream()
        .filter(p -> p.getPrice() > 1000)
        .filter(p -> p.getRating() > 4)
        .filter(p -> p.getCategory().equals("Electronics"))
        .sorted((a, b) -> Double.compare(a.getPrice(), b.getPrice()))
        .limit(10)
        .toList();
```

Developer requirement batata hai.

Framework execution organize karta hai.

Interview Questions

Q1. What is a Programming Paradigm?

Answer

Programming Paradigm ek programming style ya approach hota hai jiske through hum problems solve karte hain.

Q2. Is Java a Functional Programming Language?

Answer

No.

Java primarily Object-Oriented Programming language hai.

Java 8 ke baad Functional Programming features support karta hai.

Q3. Why was Functional Programming introduced?

Answer

- Boilerplate code reduce karne ke liye
- Readability improve karne ke liye
- Reusable behavior create karne ke liye

- Collection processing simplify karne ke liye
- Parallel processing ko easy banane ke liye

Q4. Difference between Imperative and Declarative Programming?

Answer

Imperative Programming me hum machine ko **HOW** batate hain.

Declarative Programming me hum machine ko **WHAT** batate hain.

Q5. Does Stream API remove looping?

Answer

No.

Streams looping ko remove nahi karti.

Streams **internal iteration** use karti hain.

Developer explicit loop nahi likhta.

Part 2 Summary

- Programming Paradigm ka matlab programming style hota hai.
- Java ek pure Functional Language nahi hai.
- Java 8 ne Functional Programming features introduce kiye.
- Imperative = HOW.
- Declarative = WHAT.
- Stream API declarative style ko support karti hai.
- Internal iteration aur behavior passing Java 8 ke core ideas hain.

Homework

1. Programming Paradigm kya hota hai?
2. Java ko Multi-Paradigm language kyun kaha ja sakta hai?
3. Pure Functional Language aur Java me kya difference hai?
4. Imperative aur Declarative Programming ke 5 differences likho.
5. Swiggy example ki jagah apna koi real-world example do jo Declarative Programming ko explain kare.
6. Explain why "Streams do not eliminate iteration; they hide explicit iteration."

Next Part

Lecture 1 – Part 3

Topics:

- Complete Java 8 Features Overview
- Lambda Expressions Overview
- Functional Interfaces Overview
- Stream API Overview
- Method References
- Default Methods
- Static Methods
- Optional
- Date & Time API
- Nashorn JavaScript Engine Overview